Cypress App

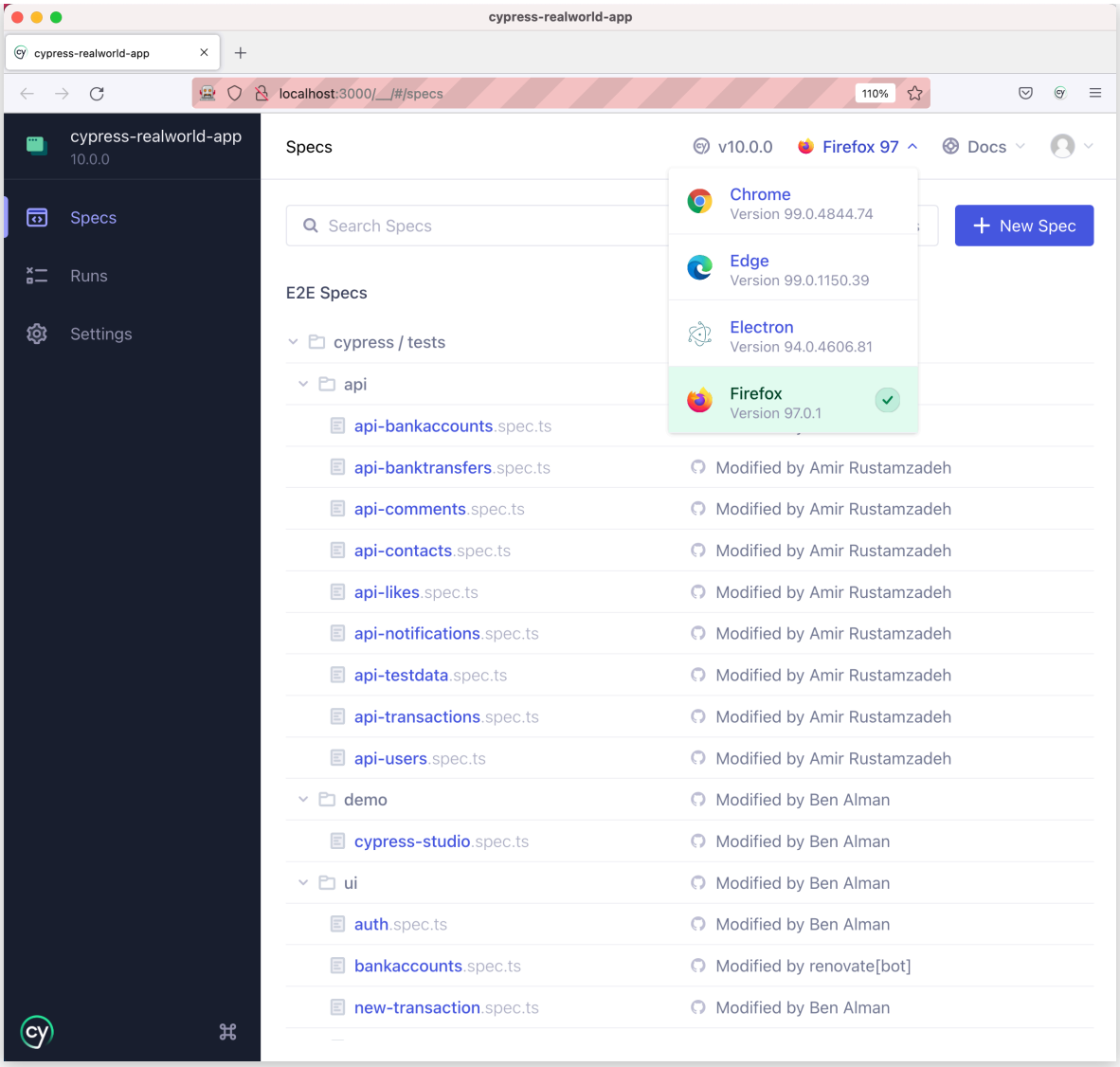
Cross Browser Testing

What you'll learn

- How to run tests across multiple browsers
- Strategies for incorporating cross-browser testing into your CI process

Cypress has the capability to run tests across multiple browsers. Currently, Cypress has support for Chrome-family browsers, Firefox, and WebKit (Safari's browser engine).

Excluding **Electron**, any browser you want to run Cypress tests in needs to be installed on your local system or CI environment. A full list of detected browsers is displayed within the browser selection menu of **Cypress**.



The desired browser can also specified via the `--browser` flag when using the `run` command to launch Cypress. For example, to run Cypress tests in Chrome:

```
cypress run --browser chrome
```

To make launching of Cypress with a specific browser even more convenient, npm scripts can be used as a shortcut:

package.json

```
"scripts": {
  "cy:run:chrome": "cypress run --browser chrome",
  "cy:run:firefox": "cypress run --browser firefox"
}
```

Continuous Integration Strategies

When incorporating testing of multiple browsers within your QA process, you must implement a CI strategy that provides an optimal level of confidence while taking into consideration test duration and infrastructure costs. This optimal strategy will vary by the needs of a particular project. This guide we present several strategies to consider when crafting the strategy for your project.

CI strategies will be demonstrated using the **Circle CI Cypress Orb** for its concise and readable configuration, but the same concepts apply for most CI providers.

> The CI configuration examples within this guide use **Cypress's Docker images** to provision testing environments with desired versions of Node, Chrome, and Firefox.

Periodic Basis

Generally, it is desired to run tests with each pushed commit, but it may not be necessary to do so for all browsers. For example, we can choose to run tests within Chrome for each commit, but only run Firefox on a periodic basis (i.e. nightly). The periodic frequency will depend on the scheduling of your project releases, so consider a test run frequency that is appropriate for the release schedule of your project.

> **Cron Scheduling**
> Typically CI providers allow for the scheduling of CI jobs via **cron expressions**. For example, the expression `0 0 * * *` translates to "everyday at midnight" or nightly. Helpful **online utilities** are available to assist with creation and translation of cron expressions.

The following example demonstrates a nightly CI schedule against production (`master` branch) for Firefox:

.circleci/config.yml

```
version: 2.1
orbs:
```

```
  cypress: cypress-io/cypress@6
workflows:
  nightly:
    triggers:
      - schedule:
          cron: '0 0 * * *'
          filters:
            branches:
              only:
                - master
    jobs:
      - cypress/run:
          install-browsers: true
          cypress-command: 'npx cypress run --browser firefox'
          start-command: 'npm start'
```

Production Deployment

For projects that exhibit consistently stable behavior across browsers, it may be better to run tests against additional browsers only before merging changes in the production deployment branch.

The following example demonstrates only running Firefox tests when commits are merged into a specific branch (`develop` branch in this case) so any potential Firefox issues can be caught before a production release:

.circleci/config.yml

```
version: 2.1
orbs:
  cypress: cypress-io/cypress@6
workflows:
  test_develop:
    jobs:
      - filters:
        branches:
          only:
            - develop
      - cypress/run:
```

```
          install-browsers: true
          cypress-command: 'npx cypress run --browser firefox'
          start-command: 'npm start'
```

Subset of Tests

We can choose to only run a subset of tests against a given browser. For example, we can execute only the happy or critical path related test files, or a directory of specific "smoke" test files. It's not always necessary to have both browsers always running *all* tests.

In the example below, the Chrome `cypress/run` job runs *all* tests against Chrome and reports results to **Cypress Cloud** using a (**group**) named `chrome`.

The Firefox `cypress/run` job runs a subset of tests, defined in the `spec` parameter, against the Firefox browser, and reports the results to **Cypress Cloud** under the group `firefox-critical-path`.

> **Note:** The `name` under each `cypress/run` job which will be shown in the Circle CI workflow UI to distinguish the jobs.

.circleci/config.yml

```
version: 2.1
orbs:
  cypress: cypress-io/cypress@6
workflows:
  build:
    jobs:
      - cypress/run:
          name: Chrome
          start-command: 'npm start'
          install-browsers: true
          cypress-command: 'npx cypress run --browser chrome --
record --group chrome'
      - cypress/run:
          name: Firefox
```

```
          start-command: 'npm start'
          cypress-command: 'npx cypress run --browser firefox --record --group
            firefox-critical-path --spec
            cypress/e2e/signup.cy.js,cypress/e2e/login.cy.js'
```

Parallelize per browser

Execution of test files can be parallelized on a per **group** basis, where test files can be grouped by the browser under test. This versatility enables the ability to allocate the desired amount of CI resources towards a browser to either improve test duration or to minimize CI costs.

You do not have to run all browsers at the same parallelization level. In the example below, the Chrome dedicated `cypress/run` job runs *all* tests in parallel, across **4 machines**, against Chrome and reports results to **Cypress Cloud** under the group name `chrome`. The Firefox dedicated `cypress/run` job runs a *subset* of tests in parallel, across **2 machines**, defined by the `spec` parameter, against the Firefox browser and reports results to **Cypress Cloud** under the group named `firefox`.

.circleci/config.yml

```
version: 2.1
orbs:
  cypress: cypress-io/cypress@6
workflows:
  build:
    jobs:
      - cypress/run:
          name: Chrome
          cypress-command: 'npx cypress run --record --parallel --group chrome --browser chrome'
          start-command: 'npm start'
          parallelism: 4
          install-browsers: true
      - cypress/run:
          name: Firefox
          cypress-command:
```

```
            'npx cypress run --record --parallel --group
firefox --browser
            firefox --spec

cypress/e2e/app.cy.js,cypress/e2e/login.cy.js,cypress/e2e/about
.cy.js'
          start-command: 'npm start'
          parallelism: 2
          install-browsers: true
```

Running Specific Tests by Browser

There may be instances where it can be useful to run or ignore one or more tests when in specific browsers. For example, test run duration can be reduced by only running smoke-tests against Chrome and not Firefox. This type of granular selection of test execution depends on the type of tests and the level of confidence those specific tests provide to the overall project.

> When considering to ignore or only run a particular test within a given browser, assess the true need for the test to run on multiple browsers.

You can specify a browser to run or exclude by passing a matcher to the suite or test within the **test configuration**. The `browser` option accepts the same arguments as **Cypress.isBrowser()**.

test.cy.js

```
// Run the test if Cypress is running
// using the built-in Electron browser
it('has access to clipboard', { browser: 'electron' }, () => {
  // ...
})

// Run the test if Cypress is run via Firefox
it('Download extension in Firefox', { browser: 'firefox' }, ()
=> {
  cy.get('#dl-extension').should('contain', 'Download Firefox
```

```
Extension')
})

// Run happy path tests if Cypress is run via Firefox
describe('happy path suite', { browser: 'firefox' }, () => {
  it('...')
})

// Ignore test if Cypress is running via Chrome
// This test is not recorded to Cypress Cloud
it('Show warning outside Chrome', { browser: '!chrome' }, () =>
{
  cy.get('.browser-warning').should(
    'contain',
    'For optimal viewing, use Chrome browser'
  )
})
```

See also

- Browser Launch API
- Cypress.browser
- Cypress.isBrowser
- Launching Browsers
- Test Configuration

*Last updated on **Nov 11, 2025***

CONTENTS